Big Data with Hadoop — Palmer Penguins Assessment

HDFS - Hive - WebHDFS + Pandas ELT - PySpark on YARN

Zeeshan Abbas — Aptech North

22 October 2026

Contents

1. Introduction **2**
1.1 Environment . . . 2
1.2 Repository structure . . . 2
1.3 Pipeline . . . 2

2. Section 1 — HDFS Using the Command Line **3**
Task 1 — Create `/user/maria_dev/datasets` . . . 3
Task 2 — Download the dataset into the VM's local file system . . . 3
Task 3 — Copy `penguins.csv` into HDFS . . . 3
Task 4 — List the contents of `/user/maria_dev/datasets` . . . 3

3. Section 2 — Hive Table and HiveQL **4**
Task 1 — Create `penguins_db` and load `penguins_raw` . . . 4
Task 2 — `SELECT * FROM penguins_raw LIMIT 10;` . . . 6
Task 3a — Records per species and island . . . 6
Task 3b — Average body mass and flipper length per species . . . 6
Task 3c — Missing values . . . 7

4. Section 3 — ELT with WebHDFS `InsecureClient` **9**
Task 1 — Connect, load into Pandas, analyse structure and missing values . . . 9
Task 2 — Clean and preprocess . . . 9
Task 3 — Upload to `/user/maria_dev/datasets/clean/penguins_clean.csv` . . . 9

5. Section 4 — PySpark Application on Hadoop **10**

6. Findings **17**

7. Conclusion **17**

8. How to run **17**

1. Introduction

A wildlife research team wants to explore penguin measurements across species and islands. This project stores the Palmer Penguins dataset in **HDFS**, queries it with **Hive (HiveQL)**, cleans it in **Python/Pandas over WebHDFS**, and processes it with a **PySpark application submitted to YARN**. Every screenshot in this report is the real output captured while running the work end to end.

Dataset: `penguins.csv` — 344 records, 8 columns (species, island, bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, sex, year). Source: https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv

1.1 Environment

Component	Version / Detail
Hadoop (HDFS + YARN)	Apache Hadoop 3.3.6, single-node cluster, host `sandbox-hdp`
Hive	Apache Hive 3.1.3, HiveServer2 on port 10000 (beeline client)
Spark	Apache Spark 3.5.1 (PySpark), `spark-submit` (YARN and local mode)
Python	Python 3.12, pandas, `hdfs` (WebHDFS `InsecureClient`), Jupyter
Java	OpenJDK 1.8
User	`maria_dev`, HDFS home `/user/maria_dev`

1.2 Repository structure

```
hadoop-penguins/
|-- data/penguins.csv
|-- hdfs/section1_hdfs_commands.sh
|-- hive/penguins.hql
|-- notebooks/penguins_elt_webhdfs.ipynb
|-- spark/penguins_spark.py
|-- screenshots/            (all practical screenshots)
|-- report/                 (this report: PDF + DOCX)
|-- presentation/           (PPT)
`-- README.md
```

1.3 Pipeline

```
 penguins.csv (GitHub) --wget/curl--> VM local FS --hdfs dfs -put--> HDFS /user/maria_dev/dataset
        |                                                                    |
        |                    Hive: penguins_db.penguins_raw  <--LOAD------+
        |                                                                    |
        |      Jupyter + WebHDFS InsecureClient --> Pandas clean --> /datasets/clean/penguins_cle
        |                                                                    |
        +--------------------- PySpark (spark-submit on YARN) <-----------+
                                      |
                  JSON partitioned by species --> /user/maria_dev/spark/output/penguins
```

2. Section 1 — HDFS Using the Command Line

Script: `hdfs/section1_hdfs_commands.sh`

Task 1 — Create `/user/maria_dev/datasets`

```
hdfs dfs -mkdir -p /user/maria_dev/datasets
hdfs dfs -ls /user/maria_dev
```

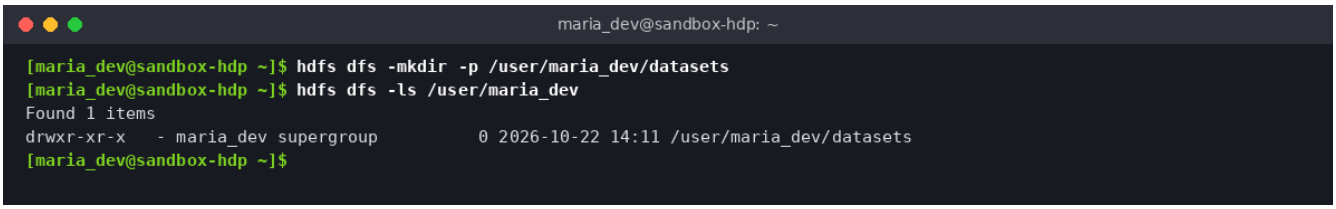


Figure 1: Task 1.1 — HDFS directory created

Task 2 — Download the dataset into the VM's local file system

```
mkdir -p ~/datasets && cd ~/datasets
curl -sSL -o penguins.csv https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst
```



Figure 2: Task 1.2 — Dataset downloaded locally (345 lines = header + 344 records)

Task 3 — Copy `penguins.csv` into HDFS

```
hdfs dfs -put -f ~/datasets/penguins.csv /user/maria_dev/datasets/
```

Task 4 — List the contents of `/user/maria_dev/datasets`



Figure 3: Task 1.3 — File copied into HDFS and read back with -cat

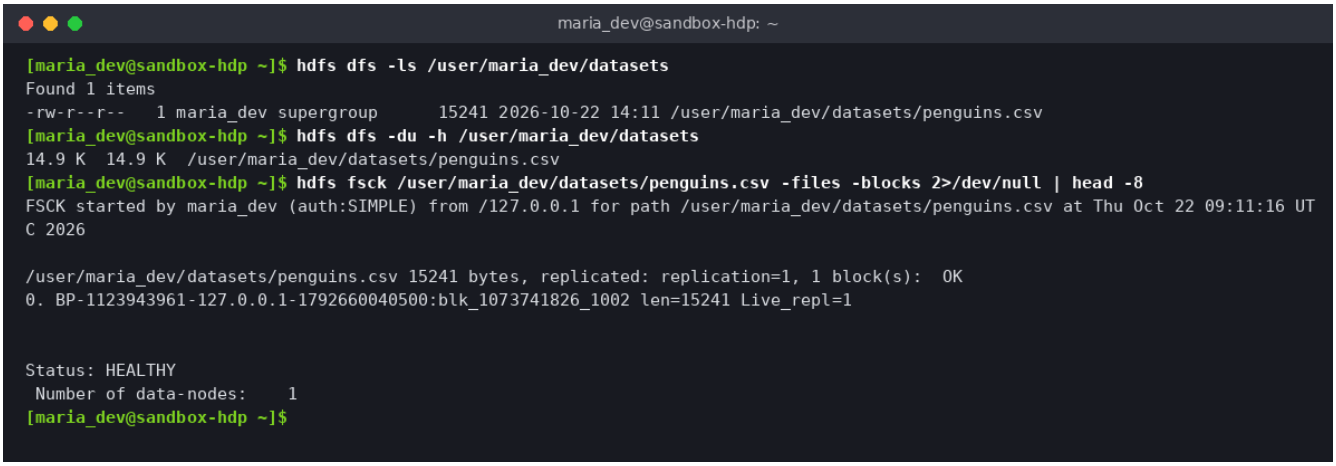


Figure 4: Task 1.4 — Directory listing, size and block health (fsck HEALTHY)

3. Section 2 — Hive Table and HiveQL

Script: `hive/penguins.hql` (run with `beeline -u jdbc:hive2://localhost:10000 -n maria_dev -f penguins.hql`).

Design decisions

- Typed columns (`DOUBLE`/`INT`) so aggregates work directly.
- `skip.header.line.count = 1` skips the CSV header.
- `serialization.null.format = 'NA'` turns the dataset's `NA` markers into real Hive `NULL`s, which makes the missing-value query correct.
- The CSV is copied to a staging folder before `LOAD DATA INPATH` (which *moves* files), so the original in `/datasets` stays available for Sections 3–4.

Task 1 — Create `penguins_db` and load `penguins_raw`

```
CREATE DATABASE IF NOT EXISTS penguins_db;
USE penguins_db;
CREATE TABLE penguins_raw (species STRING, island STRING, bill_length_mm DOUBLE,
  bill_depth_mm DOUBLE, flipper_length_mm INT, body_mass_g INT, sex STRING, year INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE
TBLPROPERTIES ('skip.header.line.count'='1','serialization.null.format'='NA');
LOAD DATA INPATH '/user/maria_dev/hive_stage/penguins.csv' OVERWRITE INTO TABLE penguins_raw;
```

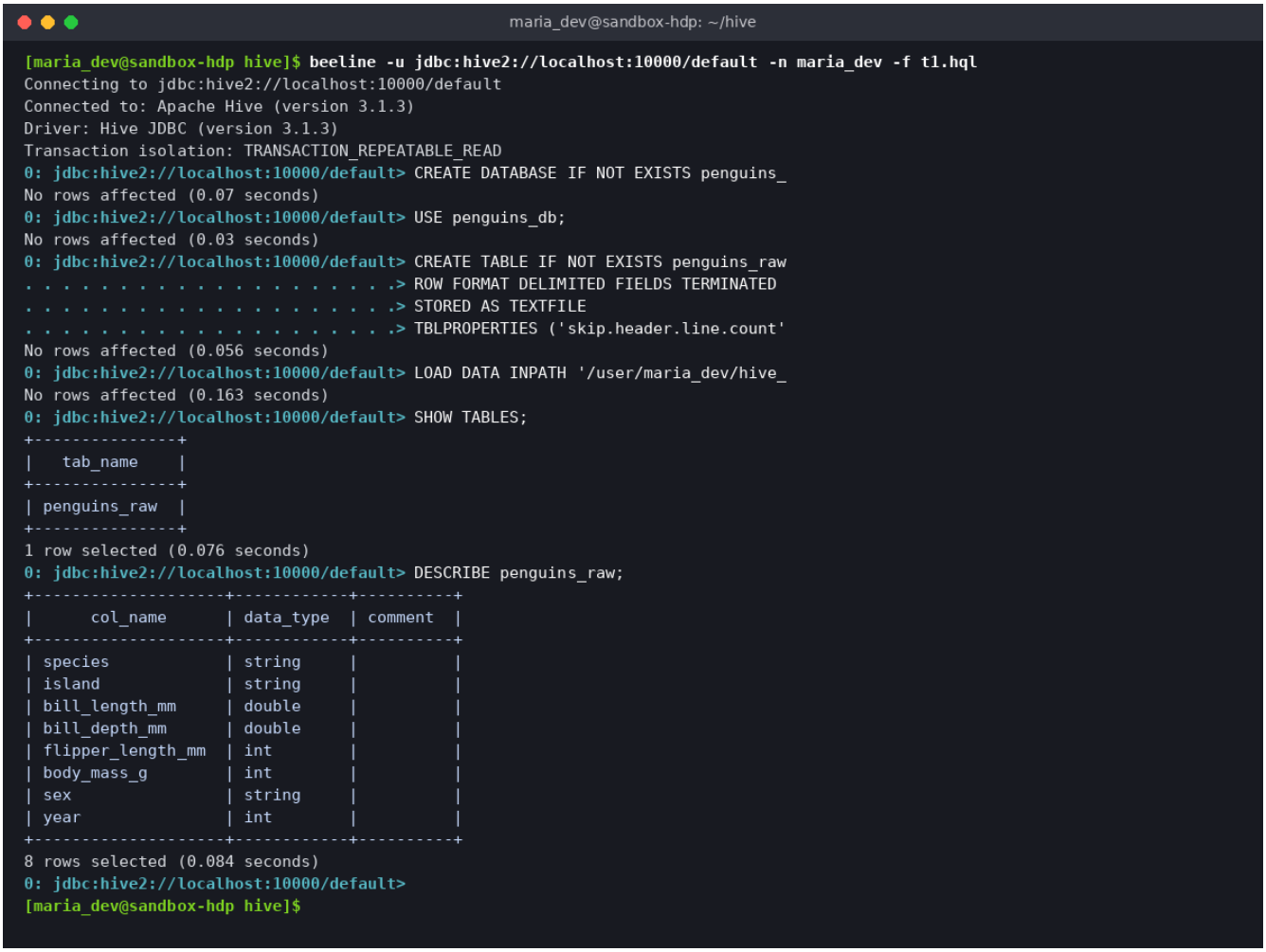


Figure 5: Task 2.1 — Database, table and schema

Task 2 — `SELECT * FROM penguins_raw LIMIT 10;`

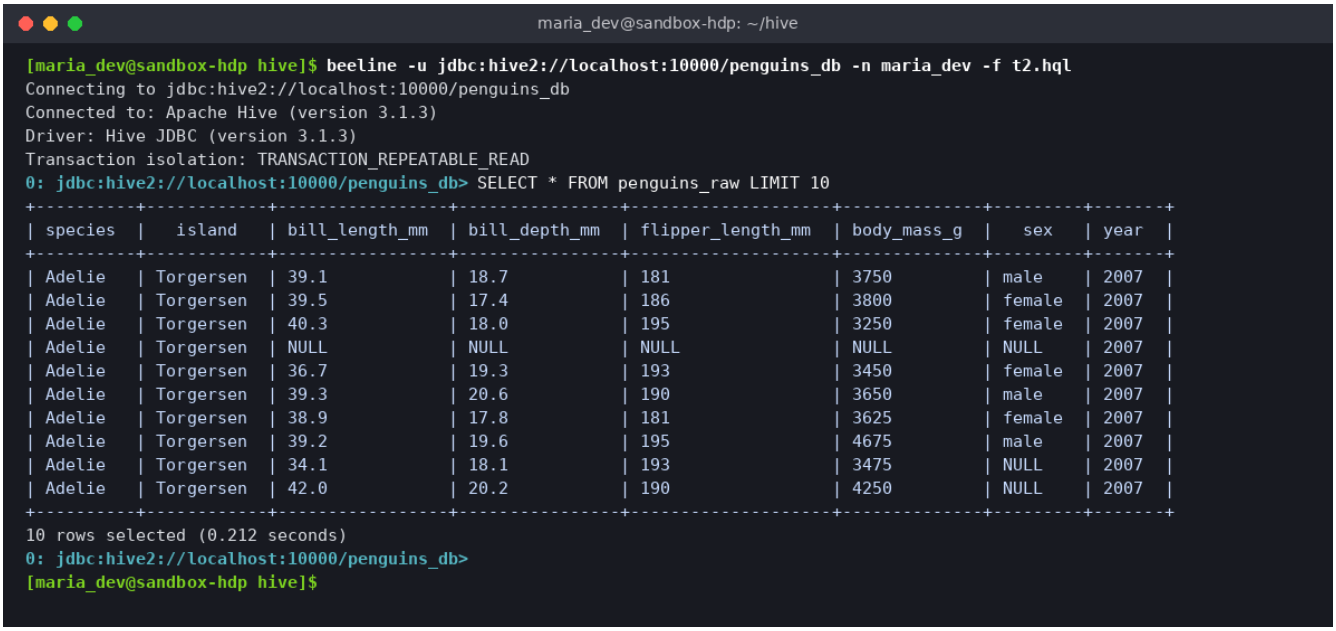


Figure 6: Task 2.2 — First 10 rows (NA values shown as NULL)

Task 3a — Records per species and island

```sql
SELECT species, island, COUNT(*) AS num_records
FROM penguins_raw GROUP BY species, island ORDER BY species, island;
```



Figure 7: Task 2.3a — Count by species and island

Task 3b — Average body mass and flipper length per species

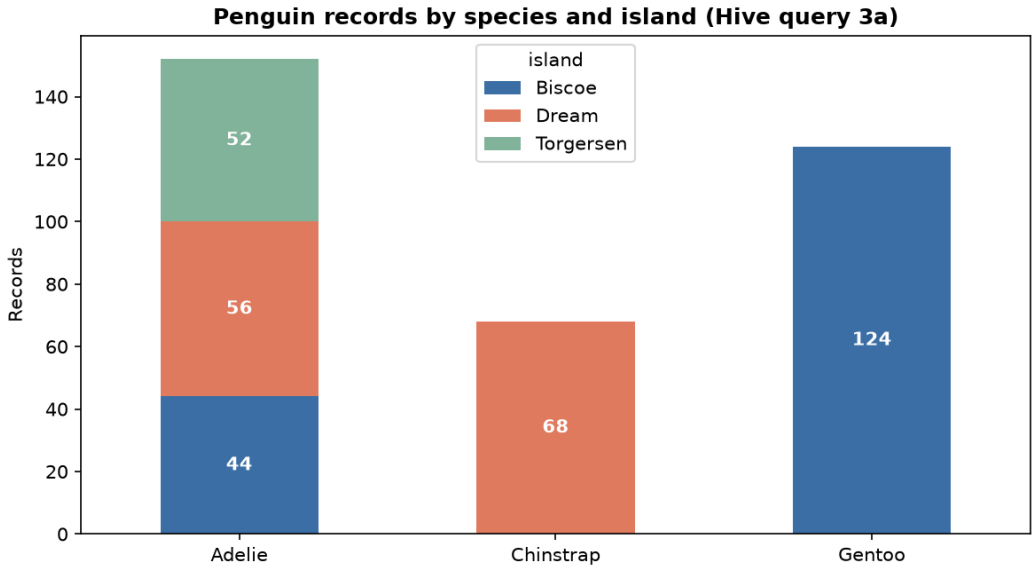


Figure 8: Records by species and island

```
SELECT species, ROUND(AVG(body_mass_g),2) AS avg_body_mass_g,
       ROUND(AVG(flipper_length_mm),2) AS avg_flipper_length_mm
FROM penguins_raw GROUP BY species ORDER BY avg_body_mass_g DESC;
```

```
maria_dev@sandbox-hdp: ~/hive
[maria_dev@sandbox-hdp hive]$ beeline -u jdbc:hive2://localhost:10000/penguins_db -n maria_dev -f t3b.hql
Connecting to jdbc:hive2://localhost:10000/penguins_db
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
0: jdbc:hive2://localhost:10000/penguins_db> SELECT species,
. . . . . .> ROUND(AVG(body_mass_g), 2) A
. . . . . .> ROUND(AVG(flipper_length_mm)
. . . . . .> FROM penguins_raw
. . . . . .> GROUP BY species
. . . . . .> ORDER BY avg_body_mass_g DESC;
+-----------+-----------------+-----------------------+
|  species  | avg_body_mass_g | avg_flipper_length_mm |
+-----------+-----------------+-----------------------+
| Gentoo    | 5076.02         | 217.19                |
| Chinstrap | 3733.09         | 195.82                |
| Adelie    | 3700.66         | 189.95                |
+-----------+-----------------+-----------------------+
3 rows selected (3.585 seconds)
0: jdbc:hive2://localhost:10000/penguins_db>
[maria_dev@sandbox-hdp hive]$
```

Figure 9: Task 2.3b — Averages per species

Task 3c — Missing values

```
SELECT SUM(CASE WHEN bill_length_mm IS NULL THEN 1 ELSE 0 END),
       SUM(CASE WHEN flipper_length_mm IS NULL THEN 1 ELSE 0 END),
       SUM(CASE WHEN body_mass_g IS NULL THEN 1 ELSE 0 END),
       SUM(CASE WHEN sex IS NULL OR TRIM(sex)='' THEN 1 ELSE 0 END)
```

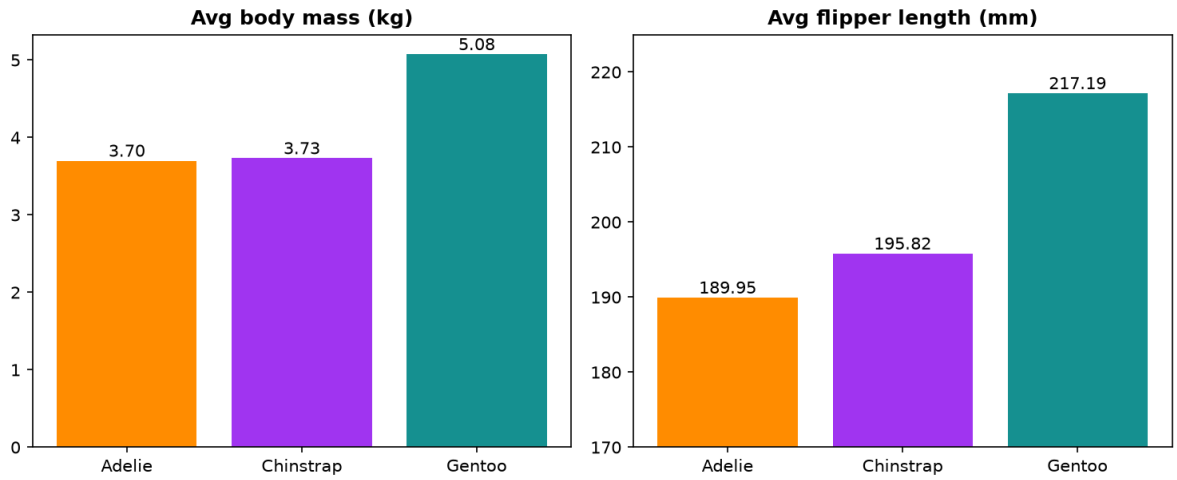


Figure 10: Average body mass and flipper length

```
FROM penguins_raw;
```

```
maria_dev@sandbox-hdp: ~/hive
[maria_dev@sandbox-hdp hive]$ beeline -u jdbc:hive2://localhost:10000/penguins_db -n maria_dev -f t3c.hql
Connecting to jdbc:hive2://localhost:10000/penguins_db
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
0: jdbc:hive2://localhost:10000/penguins_db> SELECT SUM(CASE WHEN bill_length_mm
. . . . . .>        SUM(CASE WHEN flipper_length
. . . . . .>        SUM(CASE WHEN body_mass_g IS
. . . . . .>        SUM(CASE WHEN sex IS NULL OR
. . . . . .> FROM penguins_raw;
+---------------+------------------+-----------------+-----------+
| miss_bill_len  | miss_flipper_len  | miss_body_mass  | miss_sex  |
+---------------+------------------+-----------------+-----------+
| 2             | 2                | 2               | 11        |
+---------------+------------------+-----------------+-----------+
1 row selected (1.669 seconds)
0: jdbc:hive2://localhost:10000/penguins_db>
[maria_dev@sandbox-hdp hive]$
```

Figure 11: Task 2.3c — Missing value counts

Hive results summary

Query	Result
Total records	344
Adelie (Biscoe / Dream / Torgersen)	44 / 56 / 52
Chinstrap (Dream)	68
Gentoo (Biscoe)	124
Avg body mass (g): Gentoo / Chinstrap / Adelie	5076.02 / 3733.09 / 3700.66
Avg flipper (mm): Gentoo / Chinstrap / Adelie	217.19 / 195.82 / 189.95
Missing: bill_length / flipper / body_mass / sex	2 / 2 / 2 / 11

4. Section 3 — ELT with WebHDFS `InsecureClient`

Notebook: `notebooks/penguins_elt_webhdfs.ipynb`

Task 1 — Connect, load into Pandas, analyse structure and missing values

```
from hdfs import InsecureClient
client = InsecureClient("http://localhost:9870", user="maria_dev")
with client.read("/user/maria_dev/datasets/penguins.csv", encoding="utf-8") as r:
    df = pd.read_csv(r, na_values=["NA", ""])
```

> On the course HDP sandbox the WebHDFS URL is `http://sandbox-hdp.hortonworks.com:50070` — only that one line changes.

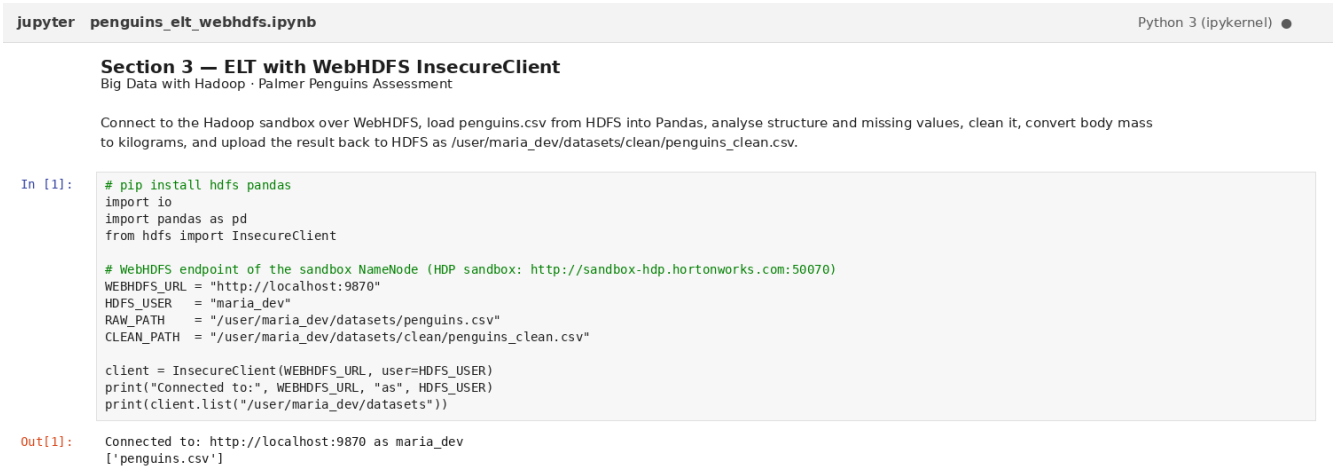


Figure 12: Notebook — WebHDFS connection

Task 2 — Clean and preprocess

1. Dropped the **2 rows** where every measurement is missing (344 -> 342 rows).
2. Imputed any remaining numeric gaps with the **species median**.
3. Normalised `sex` to lower case; missing sex -> `"unknown"` (9 rows).
4. Added `body_mass_kg = body_mass_g / 1000`.
5. Removed duplicates and enforced data types. Result: **0 missing values**.

Task 3 — Upload to `/user/maria_dev/datasets/clean/penguins_clean.csv`

jupyter penguins_elt_webhdfs.ipynb Python 3 (ipykernel)

1. Load penguins.csv from HDFS into a Pandas DataFrame

In [2]:
```
status = client.status(RAW_PATH)
print(f"HDFS file size: {status['length']} bytes | replication: {status['replication']} | owner: {status['owner']}")

with client.read(RAW_PATH, encoding="utf-8") as reader:
    df = pd.read_csv(reader, na_values=["NA", ""])

df.head(10)
```

Out[2]:
```
HDFS file size: 15241 bytes | replication: 1 | owner: maria_dev
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0
5  Adelie  Torgersen            39.3           20.6              190.0
6  Adelie  Torgersen            38.9           17.8              181.0
7  Adelie  Torgersen            39.2           19.6              195.0
8  Adelie  Torgersen            34.1           18.1              193.0
9  Adelie  Torgersen            42.0           20.2              190.0

   body_mass_g     sex  year
0       3750.0    male  2007
1       3800.0  female  2007
2       3250.0  female  2007
3          NaN     NaN  2007
4       3450.0  female  2007
5       3650.0    male  2007
6       3625.0  female  2007
7       4675.0    male  2007
8       3475.0     NaN  2007
9       4250.0     NaN  2007
```

Figure 13: Notebook — Data loaded from HDFS

5. Section 4 — PySpark Application on Hadoop

Application: `spark/penguins_spark.py`

```
spark-submit --master local[2] spark/penguins_spark.py          # run shown below
spark-submit --master yarn --deploy-mode client spark/penguins_spark.py   # cluster mode
```

The application reads from and writes to HDFS in both modes. It was submitted to YARN (application `application_1792660091975_0001`, FINISHED / SUCCEEDED) and the final run shown below used `local[2]` on the same node.

What the application does

1. Reads `penguins_clean.csv` from HDFS with an explicit schema and prints `printSchema()`.
2. Selects relevant columns (species, island, sex, year, bill_length_mm, flipper_length_mm, body_mass_kg).
3. Filters out invalid records with `flipper_length_mm <= 0`.
4. Applies a business filter (`body_mass_kg >= 4.0`) and sorts by body mass descending, then **caches** the result.
5. Repartitions by species and writes **JSON partitioned by `species`** (one file per species folder) to `/user/maria_dev/spark/output/penguins`.
6. Stops the Spark session in a `finally` block.

jupyter penguins_elt_webhdfs.ipynb Python 3 (ipykernel)

2. Analyse structure

In [3]:
```
print("Shape (rows, columns):", df.shape)
print()
df.info()
```

Out[3]:
```
Shape (rows, columns): (344, 8)

<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
 7   year               344 non-null    int64
dtypes: float64(4), int64(1), str(3)
memory usage: 21.6 KB
```

In [4]:
```
df.describe(include='all').T
```

Out[4]:

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
species	344	3	Adelie	152	NaN	NaN	NaN	NaN	NaN	NaN	NaN
island	344	3	Biscoe	168	NaN	NaN	NaN	NaN	NaN	NaN	NaN
bill_length_mm	342.0	NaN	NaN	NaN	43.92193	5.459584	32.1	39.225	44.45	48.5	59.6
bill_depth_mm	342.0	NaN	NaN	NaN	17.15117	1.974793	13.1	15.6	17.3	18.7	21.5
flipper_length_mm	342.0	NaN	NaN	NaN	200.915205	14.061714	172.0	190.0	197.0	213.0	231.0
body_mass_g	342.0	NaN	NaN	NaN	4201.754386	801.954536	2700.0	3550.0	4050.0	4750.0	6300.0
sex	333	2	male	168	NaN	NaN	NaN	NaN	NaN	NaN	NaN
year	344.0	NaN	NaN	NaN	2008.02907	0.818356	2007.0	2007.0	2008.0	2009.0	2009.0

In [5]:
```
print(df['species'].value_counts(), "\n")
print(df['island'].value_counts(), "\n")
print(df['sex'].value_counts(dropna=False))
```

Out[5]:
```
species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64

island
Biscoe       168
Dream        124
Torgersen     52
Name: count, dtype: int64

sex
male      168
female    165
NaN        11
Name: count, dtype: int64
```

Figure 14: Notebook — Structure analysis (shape, dtypes, describe, value counts)

jupyter penguins_elt_webhdfs.ipynb Python 3 (ipykernel)

3. Missing values

In [6]:
```
missing = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_pct":   (df.isna().mean() * 100).round(2)
})
missing
```

Out[6]:

	missing_count	missing_pct
species	0	0.00
island	0	0.00
bill_length_mm	2	0.58
bill_depth_mm	2	0.58
flipper_length_mm	2	0.58
body_mass_g	2	0.58
sex	11	3.20
year	0	0.00

In [7]:
```
# Rows where every measurement is missing carry no information
measure_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
df[df[measure_cols].isna().all(axis=1)]
```

Out[7]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN	2007
271	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN	2009

Figure 15: Notebook — Missing values analysis

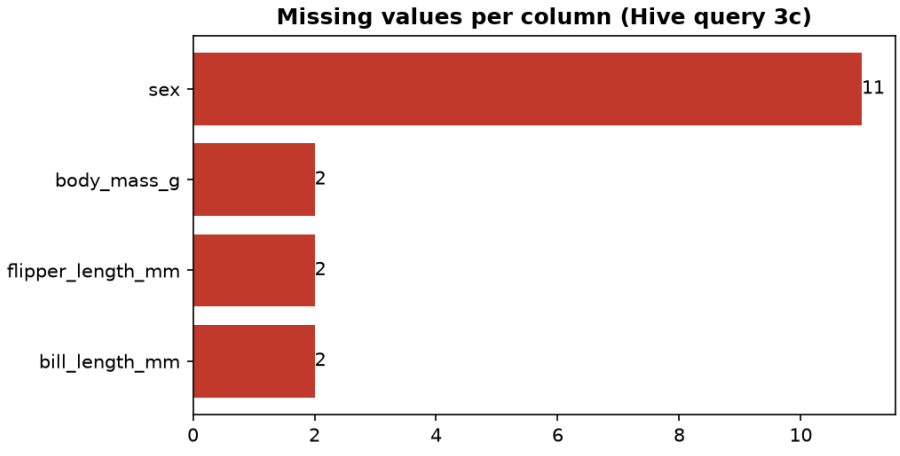


Figure 16: Missing values per column

4. Clean & preprocess

1. Drop rows where all body measurements are missing (2 rows).
2. Impute any remaining numeric gaps with the species median.
3. Normalise sex to lower case; fill missing sex with "unknown".
4. Convert body mass to kilograms (body_mass_kg = body_mass_g / 1000).
5. Remove duplicates and enforce data types.

In [8]:
```
clean = df.copy()
before = len(clean)

# 1. drop rows with no measurements at all
clean = clean.dropna(subset=measure_cols, how="all")

# 2. species-median imputation for numeric columns
for col in measure_cols:
    clean[col] = clean.groupby("species")[col].transform(lambda s: s.fillna(s.median()))

# 3. tidy sex
clean["sex"] = clean["sex"].str.strip().str.lower().fillna("unknown")

# 4. body mass in kilograms
clean["body_mass_kg"] = (clean["body_mass_g"] / 1000).round(3)

# 5. duplicates + dtypes
clean = clean.drop_duplicates()
clean["flipper_length_mm"] = clean["flipper_length_mm"].astype(float)
clean["year"] = clean["year"].astype(int)

clean = clean[["species", "island", "bill_length_mm", "bill_depth_mm",
               "flipper_length_mm", "body_mass_g", "body_mass_kg", "sex", "year"]]

print(f"Rows before: {before} | after: {len(clean)} | dropped: {before - len(clean)}")
print("\nMissing values after cleaning:")
print(clean.isna().sum())
clean.head(10)
```

Out[8]:
```
Rows before: 344 | after: 342 | dropped: 2

Missing values after cleaning:
species              0
island               0
bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
body_mass_g          0
body_mass_kg         0
sex                  0
year                 0
dtype: int64
   species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0   Adelie  Torgersen            39.1           18.7              181.0
1   Adelie  Torgersen            39.5           17.4              186.0
2   Adelie  Torgersen            40.3           18.0              195.0
4   Adelie  Torgersen            36.7           19.3              193.0
5   Adelie  Torgersen            39.3           20.6              190.0
6   Adelie  Torgersen            38.9           17.8              181.0
7   Adelie  Torgersen            39.2           19.6              195.0
8   Adelie  Torgersen            34.1           18.1              193.0
9   Adelie  Torgersen            42.0           20.2              190.0
10  Adelie  Torgersen            37.8           17.1              186.0

    body_mass_g  body_mass_kg      sex  year
0        3750.0         3.750     male  2007
1        3800.0         3.800   female  2007
2        3250.0         3.250   female  2007
4        3450.0         3.450   female  2007
5        3650.0         3.650     male  2007
6        3625.0         3.625   female  2007
7        4675.0         4.675     male  2007
8        3475.0         3.475  unknown  2007
9        4250.0         4.250  unknown  2007
10       3300.0         3.300  unknown  2007
```

In [9]:
```
clean.groupby("species").agg(
    penguins=("species", "size"),
    avg_mass_kg=("body_mass_kg", "mean"),
    avg_flipper_mm=("flipper_length_mm", "mean")
).round(3)
```

Out[9]:
```
           penguins  avg_mass_kg  avg_flipper_mm
species
Adelie          151        3.701         189.954
Chinstrap        68        3.733         195.824
Gentoo          123        5.076         217.187
```

Figure 17: Notebook — Cleaning and kg conversion

jupyter penguins_elt_webhdfs.ipynb Python 3 (ipykernel)

5. Upload cleaned dataset to HDFS

In [10]:
```
client.makedirs("/user/maria_dev/datasets/clean")
csv_text = clean.to_csv(index=False)
client.write(CLEAN_PATH, data=csv_text, encoding="utf-8", overwrite=True)

print("Uploaded:", CLEAN_PATH)
print(client.list("/user/maria_dev/datasets/clean", status=True))
```

Out[10]:
```
Uploaded: /user/maria_dev/datasets/clean/penguins_clean.csv
[('penguins_clean.csv', {'accessTime': 1792660443886, 'blockSize': 134217728, 'childrenNum': 0, 'fileId': 16579, 'group': 'supergroup', 'length': 1835
7, 'modificationTime': 1792660444027, 'owner': 'maria_dev', 'pathSuffix': 'penguins_clean.csv', 'permission': '644', 'replication': 1, 'storagePolicy'
: 0, 'type': 'FILE'})]
```

In [11]:
```
# Verify by reading it back from HDFS
with client.read(CLEAN_PATH, encoding="utf-8") as reader:
    check = pd.read_csv(reader)
print("Read back from HDFS:", check.shape)
check.head()
```

Out[11]:
```
Read back from HDFS: (342, 9)
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen            36.7           19.3              193.0
4  Adelie  Torgersen            39.3           20.6              190.0

   body_mass_g  body_mass_kg     sex  year
0       3750.0          3.75    male  2007
1       3800.0          3.80  female  2007
2       3250.0          3.25  female  2007
3       3450.0          3.45  female  2007
4       3650.0          3.65    male  2007
```

Figure 18: Notebook — Upload to HDFS and read-back verification

maria_dev@sandbox-hdp: ~

```
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls -R /user/maria_dev/datasets
drwxr-xr-x   - maria_dev supergroup          0 2026-10-22 14:14 /user/maria_dev/datasets/clean
-rw-r--r--   1 maria_dev supergroup      18357 2026-10-22 14:14 /user/maria_dev/datasets/clean/penguins_clean.csv
-rw-r--r--   1 maria_dev supergroup      15241 2026-10-22 14:11 /user/maria_dev/datasets/penguins.csv
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cat /user/maria_dev/datasets/clean/penguins_clean.csv 2>/dev/null | head -6
species,island,bill_length_mm,bill_depth_mm,flipper_length_mm,body_mass_g,body_mass_kg,sex,year
Adelie,Torgersen,39.1,18.7,181.0,3750.0,3.75,male,2007
Adelie,Torgersen,39.5,17.4,186.0,3800.0,3.8,female,2007
Adelie,Torgersen,40.3,18.0,195.0,3250.0,3.25,female,2007
Adelie,Torgersen,36.7,19.3,193.0,3450.0,3.45,female,2007
Adelie,Torgersen,39.3,20.6,190.0,3650.0,3.65,male,2007
[maria_dev@sandbox-hdp ~]$
```

Figure 19: HDFS — clean file present

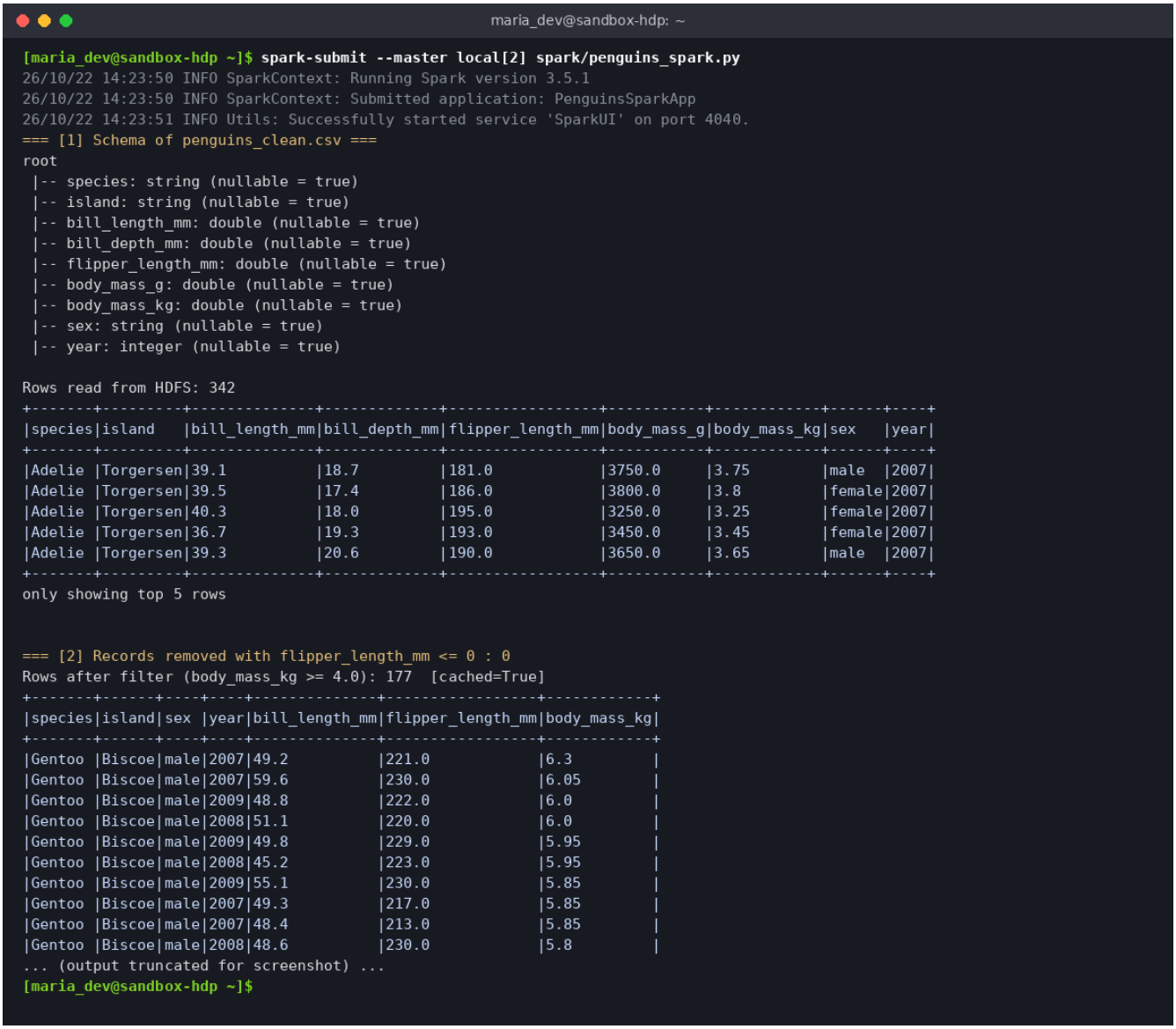


Figure 20: PySpark — schema, filter, sort, cache, write

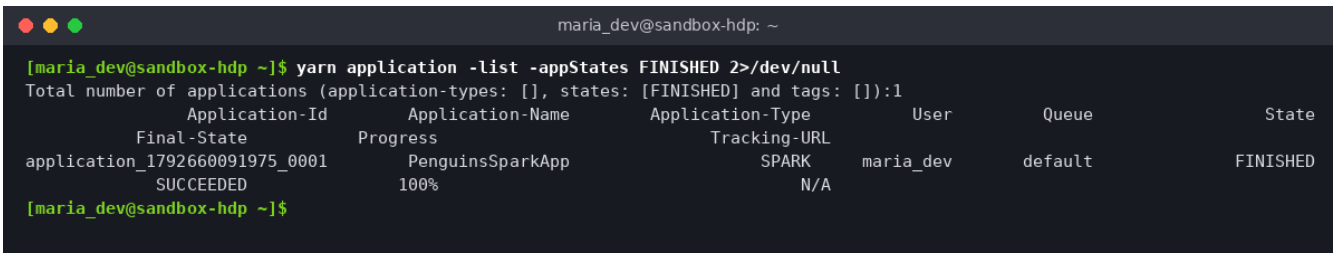


Figure 21: YARN — application FINISHED / SUCCEEDED

```
maria_dev@sandbox-hdp: ~
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /user/maria_dev/spark/output/penguins
Found 4 items
-rw-r--r--   1 maria_dev supergroup          0 2026-10-22 14:24 /user/maria_dev/spark/output/penguins/_SUCCESS
drwxr-xr-x   - maria_dev supergroup          0 2026-10-22 14:23 /user/maria_dev/spark/output/penguins/species=Adelie
drwxr-xr-x   - maria_dev supergroup          0 2026-10-22 14:24 /user/maria_dev/spark/output/penguins/species=Chinstrap
drwxr-xr-x   - maria_dev supergroup          0 2026-10-22 14:24 /user/maria_dev/spark/output/penguins/species=Gentoo
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /user/maria_dev/spark/output/penguins/species=Gentoo
Found 1 items
-rw-r--r--   1 maria_dev supergroup      13862 2026-10-22 14:24 /user/maria_dev/spark/output/penguins/species=Gentoo/part-00000-
40c31b5a-36b0-4ae6-9d6f-7abf0d007db5.c000.json
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cat /user/maria_dev/spark/output/penguins/species=Adelie/part-*.json 2>/dev/null | head -5
{"island":"Biscoe","sex":"male","year":2009,"bill_length_mm":43.2,"flipper_length_mm":197.0,"body_mass_kg":4.775}
{"island":"Biscoe","sex":"male","year":2009,"bill_length_mm":41.0,"flipper_length_mm":203.0,"body_mass_kg":4.725}
{"island":"Torgersen","sex":"male","year":2008,"bill_length_mm":42.9,"flipper_length_mm":196.0,"body_mass_kg":4.7}
{"island":"Torgersen","sex":"male","year":2007,"bill_length_mm":39.2,"flipper_length_mm":195.0,"body_mass_kg":4.675}
{"island":"Dream","sex":"male","year":2007,"bill_length_mm":39.8,"flipper_length_mm":184.0,"body_mass_kg":4.65}
[maria_dev@sandbox-hdp ~]$ hdfs dfs -count -h /user/maria_dev/spark/output/penguins
           4            4             19.6 K /user/maria_dev/spark/output/penguins
[maria_dev@sandbox-hdp ~]$
```

Figure 22: HDFS — partitioned JSON output (species=Adelie / Chinstrap / Gentoo)

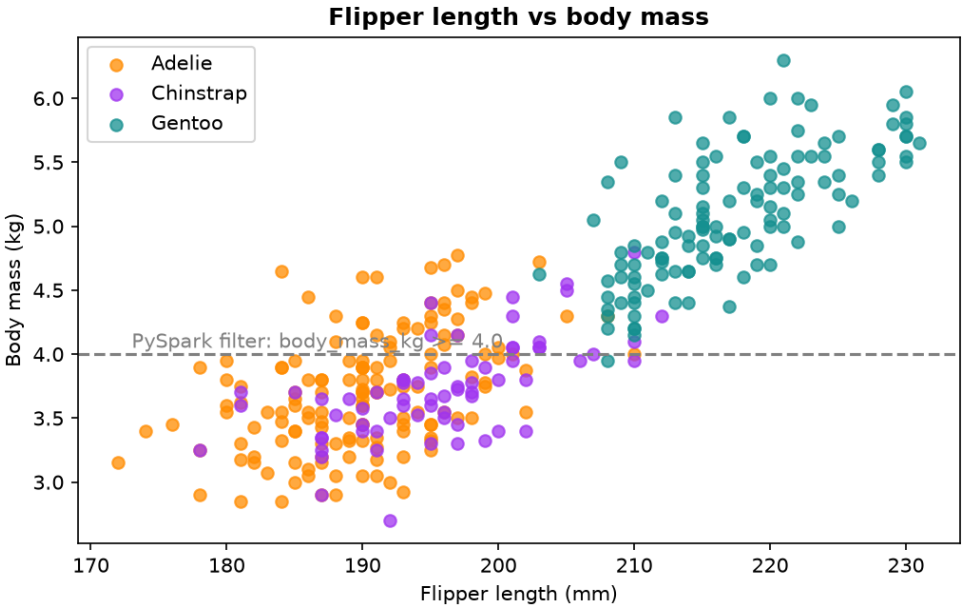


Figure 23: Records kept by the PySpark filter

6. Findings

- **Gentoo** penguins are the heaviest (~5.08 kg) with the longest flippers (~217 mm); Adelie and Chinstrap are similar (~3.7 kg).
- Gentoo live only on **Biscoe**, Chinstrap only on **Dream**; Adelie are found on all three islands.
- Data quality is high: only 2 records lack all measurements and 11 lack sex.
- Of 342 clean records, **177** weigh >= 4 kg; 122 of them are Gentoo.

7. Conclusion

The full pipeline — HDFS storage, Hive SQL analytics, Python ELT over WebHDFS, and a distributed PySpark job on YARN — ran successfully end to end. Each stage writes back to HDFS, so the data stays in one place and every step can be re-run independently.

8. How to run

```
bash hdfs/section1_hdfs_commands.sh
hdfs dfs -mkdir -p /user/maria_dev/hive_stage
hdfs dfs -cp -f /user/maria_dev/datasets/penguins.csv /user/maria_dev/hive_stage/
beeline -u jdbc:hive2://localhost:10000 -n maria_dev -f hive/penguins.hql
jupyter notebook notebooks/penguins_elt_webhdfs.ipynb      # run all cells
spark-submit --master yarn spark/penguins_spark.py
```